AWS IAM INSIDER THREAT DETECTION · PHASE 6

Student Dashboard: Plan and Tasks

What we're building, what it will look like, and who does what. This document is a plan only. Nothing here is deployed yet.

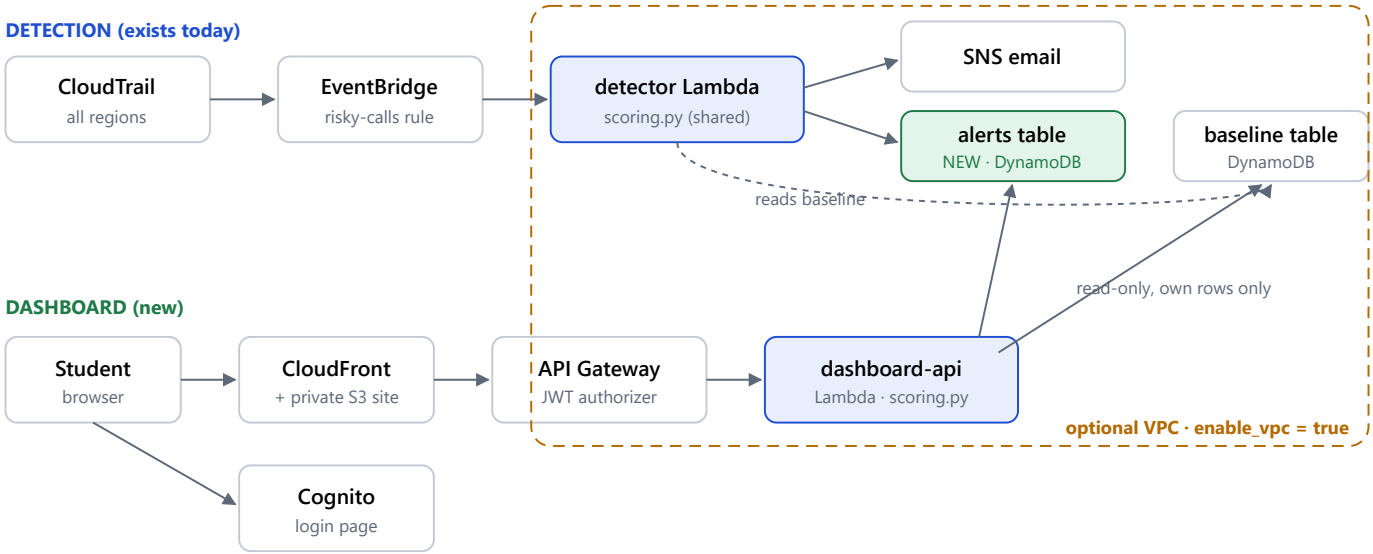
What a student gets

- **A login page.** Each account is linked to that student's IAM identity.
- **My alerts:** every alert the detector emailed, with the score, reasons and MITRE tactic.
- **What's normal for me:** their own baseline (regions, hours, IPs, APIs).
- **What triggers an alert:** the scoring rules, plus a *what-if* calculator.
- **A CLI version:** `my_baseline.py`, which uses their own `aws` login.

Decisions made

- Hosted website with a **Cognito** login page.
- **Email alerts stay.** Every event is *also* saved to DynamoDB.
- DynamoDB stays as the database.
- The VPC is **optional**, through `enable_vpc = true / false`.
- Target cost: **$0/month** with the VPC off.

Architecture



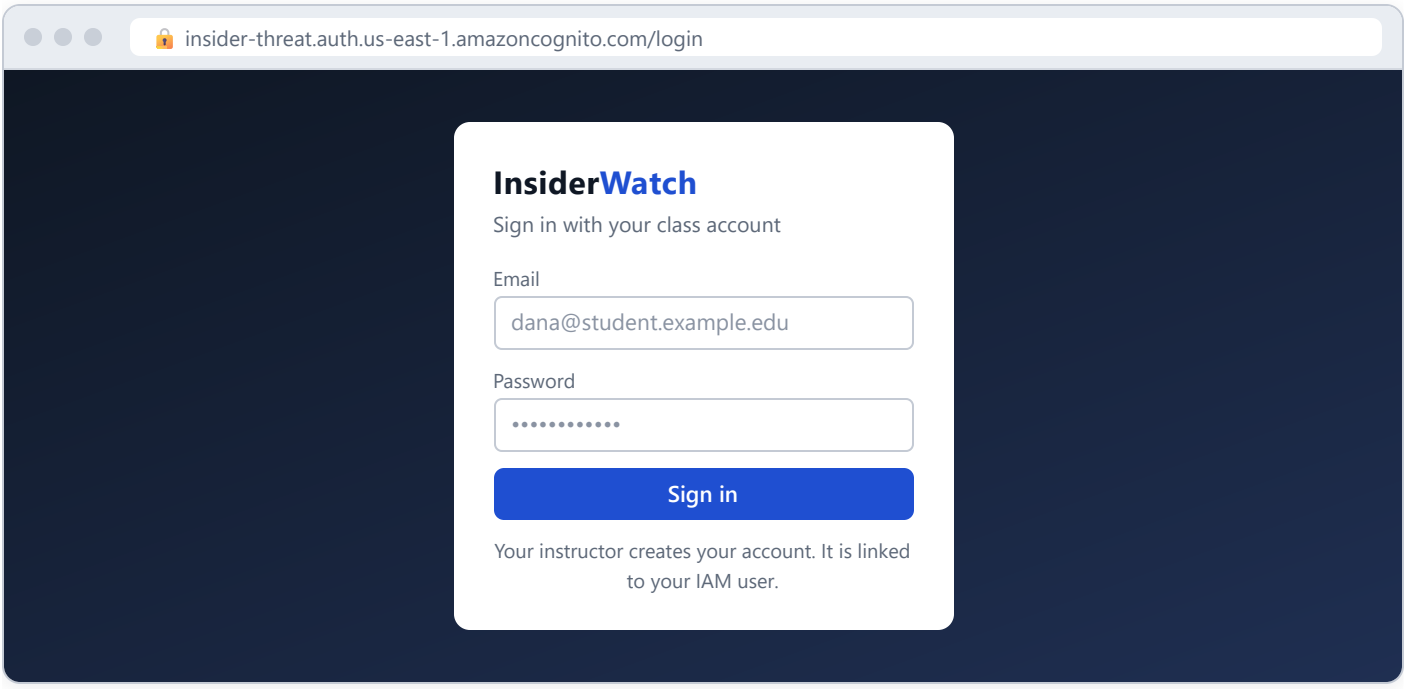
Both Lambdas import the same `scoring.py`, so the dashboard's what-if prediction always matches what the real detector would score. The dashed box is what moves into private subnets when `enable_vpc = true`.

> **Teaching vs. security:** showing students exactly what is "normal" also shows an attacker how to stay under the threshold. That's acceptable for a classroom tool, and the student guide should say so.

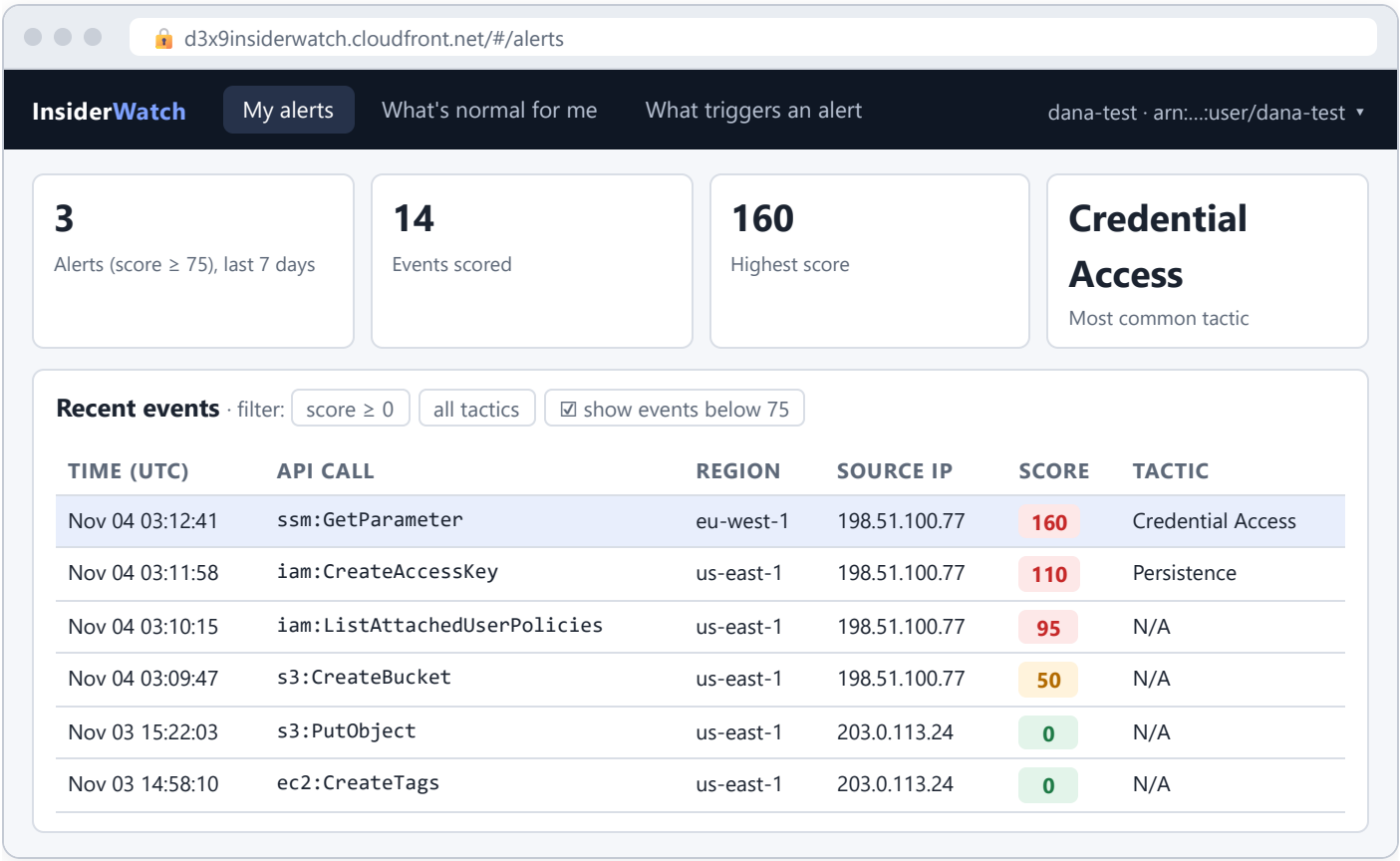
SCREEN MOCKUPS · 1 OF 4

Login page and "My alerts"

Sample data shows the dana-test persona after running `simulate_attack.py secret-theft`.



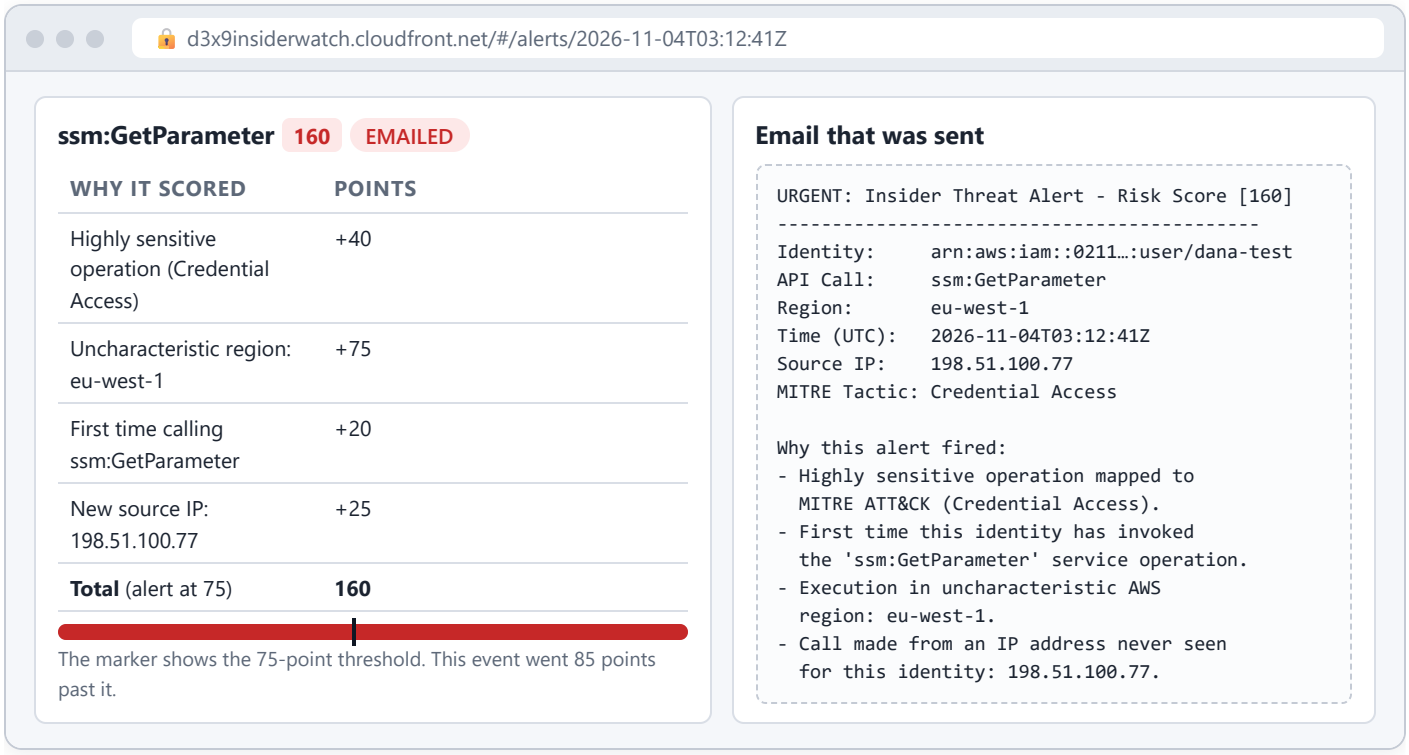
This is the Cognito managed login page. After sign-in, the student's account is linked to `custom:iam_arn`, which only an admin can set.



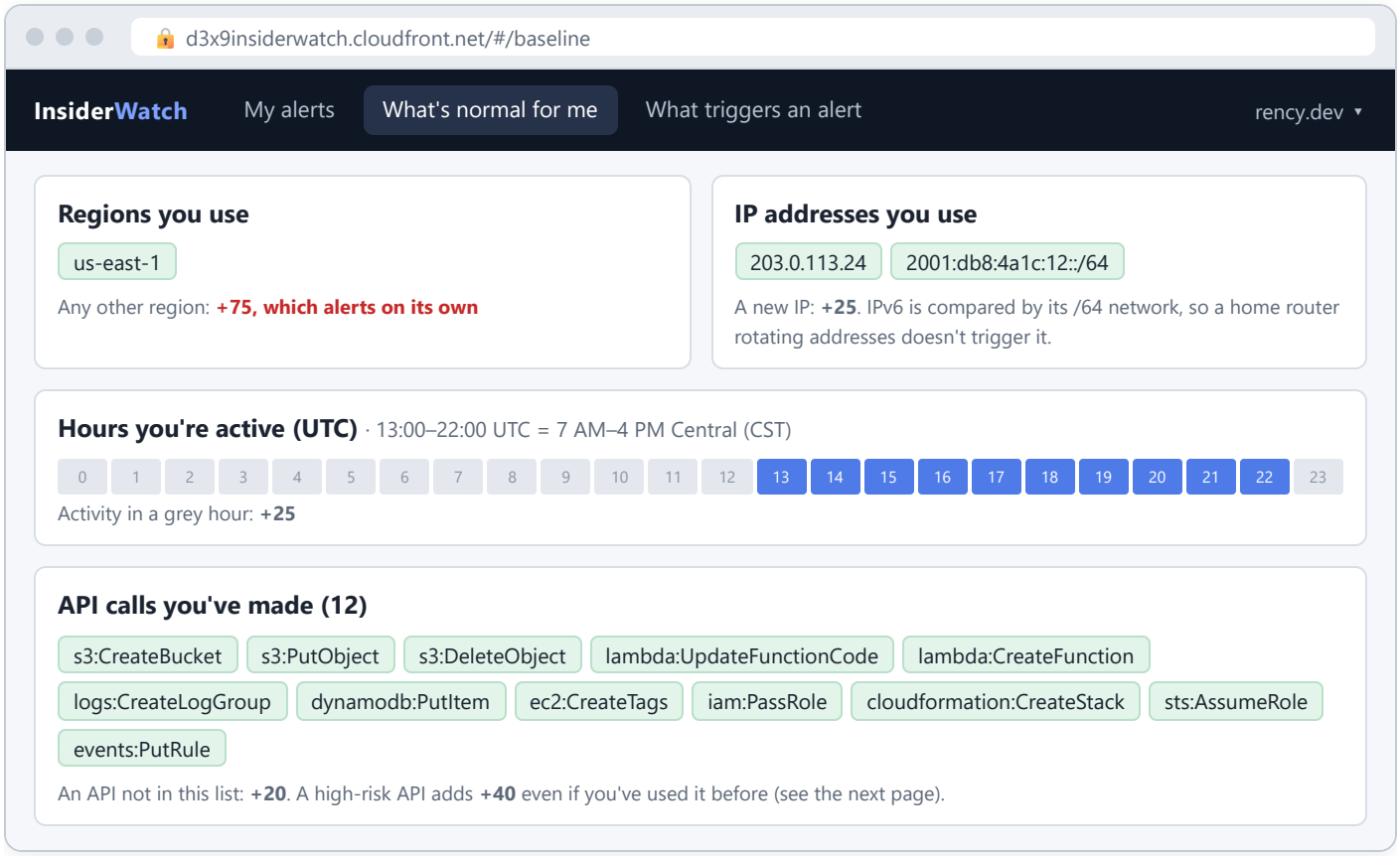
Red is 75 or more (an alert was emailed). Amber is 40–74 (close to alerting). Green is below 40. Click a row to open the detail on the next page.

SCREEN MOCKUPS · 2 OF 4

Alert detail and "What's normal for me"



The detector stores the same subject and body it emails, so the dashboard shows exactly what the student received.



This page is built from the student's `profile` row in `insider-threat-baseline`, the same row the detector reads.

SCREEN MOCKUPS · 3 OF 4

"What triggers an alert" and the what-if calculator

d3x9insiderwatch.cloudfront.net/#/rules

InsiderWatch My alerts What's normal for me What triggers an alert rency.dev

How a score is built · alert at 75

SIGNAL	POINTS
Region you've never used	**+75**
Tampering with logging or the detector	**+75**
High-risk API (list below)	+40
No baseline at all (new identity)	+40
Console login without MFA	+40
Security group opened to 0.0.0.0/0	+35
New source IP	+25
Outside your normal hours	+25
Call denied (AccessDenied)	+25
Failed console login	+25
API you've never called	+20
Any root-user activity	at least 75

What-if: "If I did this, would it alert?"

API call: `iam:CreateUser` ▾

Region: eu-west-1 ▾

Hour (UTC): 15

Source IP: 203.0.113.24 (yours)

☐ Call would be denied

Predict score

135 WOULD ALERT · Persistence

+40 high-risk API (Persistence)
+20 first time calling iam:CreateUser
+75 region eu-west-1 is not in your baseline
✓ hour 15 is normal · ✓ IP is known

This runs the same `score_event()` code the real detector uses. Nothing is sent to AWS.

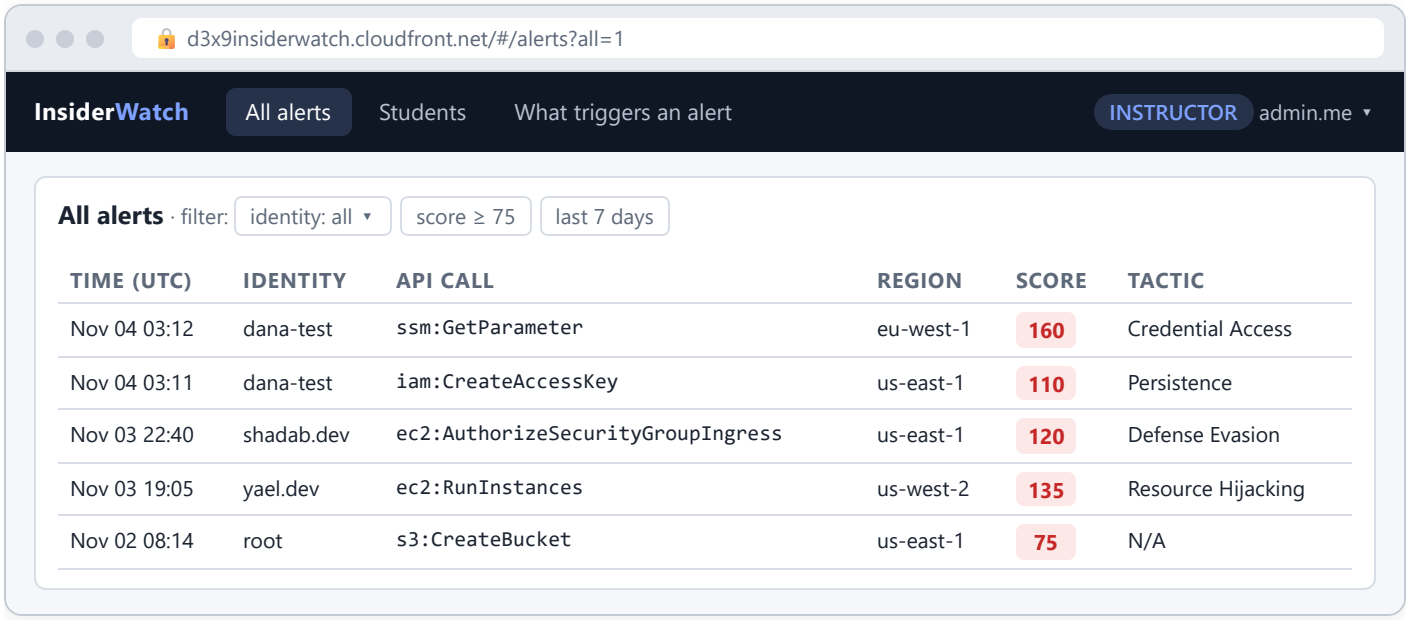
High-risk APIs (+40, or +75 for defense evasion)

TACTIC	API CALLS
Credential Access	`ssm:GetParameter(s)`, `secretsmanager:GetSecretValue`
Persistence	`iam:CreateAccessKey`, `iam:CreateUser`, `iam:CreateLoginProfile`, `iam:UpdateLoginProfile`
Privilege Escalation	`iam:AttachUserPolicy`, `iam:PutUserPolicy`, `iam:CreatePolicyVersion`, `iam:AddUserToGroup`, …
Defense Evasion	`cloudtrail:StopLogging`, `cloudtrail:DeleteTrail`, `guardduty:DeleteDetector`, `kms:ScheduleKeyDeletion`, …
Exfiltration	`s3:PutBucketPolicy`, `s3:PutBucketAcl`, `ec2:ModifySnapshotAttribute`, …
Resource Hijacking	`ec2:RunInstances`, `lambda:CreateFunction`

The rules table is generated from `HIGH_RISK_APIS` and the weights in `scoring.py`, so it can't fall out of date.

SCREEN MOCKUPS · 4 OF 4

Instructor view and the CLI version



Only users in the Cognito `instructors` group see every identity. Students only ever get their own rows, and the server enforces that from the signed token.

CLI: "log in with my aws-cli and see my baseline"

```
PS> python helpers/my_baseline.py
Signed in as arn:aws:iam::021158484652:user/rency.dev  (from your aws CLI credentials)

What's normal for you
  Regions   us-east-1                        other region      → +75 (alerts on its own)
  Hours     13–22 UTC  (7 AM–4 PM Central)   outside           → +25
  IPs       203.0.113.24, 2001:db8:4a1c:12::/64 new IP         → +25
  APIs      12 known (s3:PutObject, lambda:UpdateFunctionCode, …)  new API → +20

PS> python helpers/my_baseline.py --what-if iam:CreateUser --region eu-west-1
Predicted score: 135  → WOULD ALERT (threshold 75)
  +40  high-risk API (Persistence)
  +20  first time calling iam:CreateUser
  +75  region eu-west-1 is not in your baseline

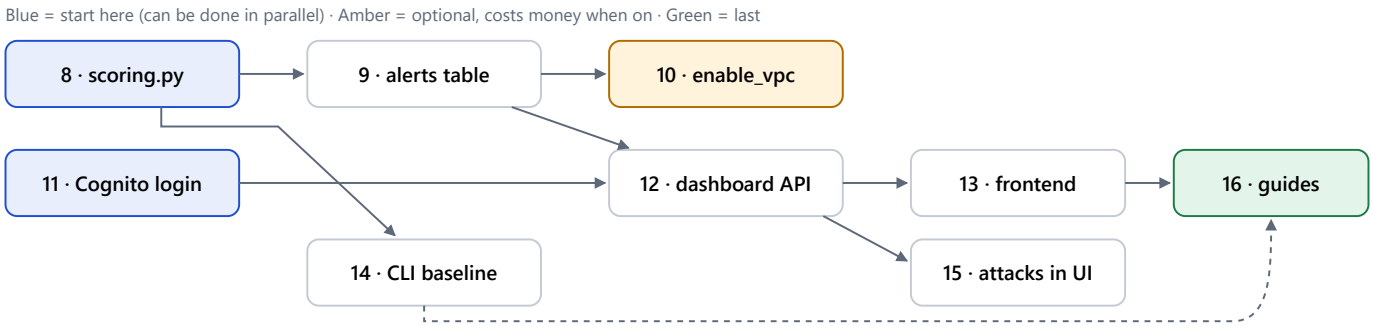
PS> python helpers/my_baseline.py --identity dana-test
AccessDenied: you can only read your own baseline.
```

An IAM policy condition (`dynamodb:LeadingKeys` = your own ARN) makes AWS itself refuse to return another student's row.

TASKS FOR THE TEAM

Who does what

Write your name on a task, create the branch, and open a PR when you've met "Done when". The order is set by the dependencies.



#	TASK	BRANCH	DEPENDS ON	COST	OWNER
8	Move scoring into `scoring.py` + explain / what-if	`task-8-scoring-module`	none	$0	
9	Alerts table; detector saves every event (email kept)	`task-9-alerts-table`	8	$0	
10	`enable_vpc` switch in Terraform	`task-10-vpc-toggle`	9	$0 off / ~$7.30 on	
11	Cognito login + link accounts to IAM users	`task-11-cognito-login`	none	~$0	
12	Dashboard API (`/me /alerts /baseline /rules /what-if`)	`task-12-dashboard-api`	8, 9, 11	~$0	
13	Dashboard frontend (S3 + CloudFront)	`task-13-dashboard-ui`	12	~$0	
14	CLI `my_baseline.py` + self-read IAM policy	`task-14-cli-baseline`	8	$0	
15	Run attack scenarios from the dashboard	`task-15-attack-runner`	12	$0	
16	Student and instructor guides	`task-16-guides`	13, 14	$0	

Cost summary

ITEM	MONTHLY COST
Cognito (under 10,000 monthly users), HTTP API, CloudFront, S3, on-demand DynamoDB, Lambda	**~$0** (free tier)
`enable_vpc = false` (default)	**$0**
`enable_vpc = true`: SNS interface endpoint in 1 AZ (needed because email is kept)	~$7.30 + $0.01/GB
DynamoDB gateway endpoint, VPC, subnets, security groups	$0

> Remember to `terraform destroy` or set `enable_vpc = false` when the VPC isn't needed. Check Billing → Bills monthly. A forgotten KMS key cost us $1 in October.

TASK DETAILS · 1 OF 3

8 Move scoring into a shared module

BRANCH	DEPENDS ON	COST	OWNER
`task-8-scoring-module`	none	$0	

Both the detector and the dashboard need to score events the same way. Move the scoring out of `lambda/handler.py` into a pure module with no AWS calls.

- New `lambda/scoring.py`: the weights, `HIGH_RISK_APIS`, the helpers (`ip_key`, `is_denied`, `resolve_identity`, ...), and `score_event(detail, baseline)` → `(score, reasons, tactics, api_call)`. Move the code without changing any logic.
- Add `explain_baseline(profile)` (what's normal, plus the points each new value adds) and `what_if(profile, api, region, hour, ip, denied)`.
- `handler.py` keeps the DynamoDB lookup, SNS and logging, and calls `score_event`.
- `main/eventbridge.tf`: change the archive from `source_file` to `source_dir = "../lambda"`.
- New `tests/test_scoring.py`.

Done when: `python -m unittest -v tests/test_handler.py tests/test_scoring.py` passes, with the Task 3/4/5 tests unchanged, and `what_if(rency, "iam:CreateUser", "eu-west-1", ...)` returns 135.

9 Alerts table: the detector saves every event (email kept)

BRANCH	DEPENDS ON	COST	OWNER
`task-9-alerts-table`	8	$0	

- `main/dynamodb.tf`: `insider-threat-alerts`, on-demand. PK `identity`, SK `sk = eventTime#eventID`. GSI `by-alerted` for the instructor view. TTL `expires_at` (30 days).
- `handler.py`: `put_item` every scored event, including the email `subject`/`body`. If the write fails, log it and still send the email.
- IAM: add `dynamodb:PutItem` on the alerts table. Add env `ALERTS_TABLE`. **Keep** `sns:Publish`.

Done when: `simulate_attack.py secret-theft` sends the email *and* creates matching rows in the alerts table.

10 VPC on/off switch: `enable_vpc`

BRANCH	DEPENDS ON	COST	OWNER
`task-10-vpc-toggle`	9	$0 off · ~$7.30/mo on	

`variable "enable_vpc" { type = bool, default = false }`. With `false`, the stack is identical to today's.

- `main/vpc.tf`, every resource with `count = var.enable_vpc ? 1 : 0`: a VPC, 2 private subnets, a route table with **no internet route**, a DynamoDB **gateway** endpoint (free), an **SNS interface endpoint** in 1 subnet (for email, ~$7.30), and a Lambda security group.
- The detector, watchdog and dashboard-api Lambdas get `dynamic "vpc_config" { for_each = var.enable_vpc ? [1] : [] }` and ENI permissions only when the switch is on.
- Usage: `terraform apply -var enable_vpc=true`.

Done when: `plan -var enable_vpc=false` shows no changes to today's stack, and with `true` an attack still emails and saves an alert. Note that turning it off takes a few minutes while Lambda releases its network interfaces.

11 Cognito login, linked to IAM users

BRANCH	DEPENDS ON	COST	OWNER
`task-11-cognito-login`	none	~$0	

- `main/dashboard.tf`: a user pool (email sign-in, managed login page) and an `instructors` group.
- Custom attribute `custom:iam_arn`, which is **admin-write only**. The app client must not be able to write it.
- An app client with the authorization-code + PKCE flow. The callback is the CloudFront URL.
- `helpers/add_student.py <email> <iam-user>` creates the user, sets the ARN and emails a temporary password.

Done when: a new student can sign in, and their ID token contains the correct `custom:iam_arn`.

12 Dashboard API

BRANCH	DEPENDS ON	COST	OWNER
`task-12-dashboard-api`	8, 9, 11	~$0	

- An API Gateway HTTP API with a Cognito JWT authorizer, and a new Lambda `lambda/dashboard_api.py` that imports `scoring.py`.
- Routes: `GET /me`, `GET /alerts`, `GET /alerts/{sk}`, `GET /baseline`, `GET /rules`, `POST /what-if`.
- The identity **always** comes from the verified token claims, never from the request. Instructors (by group claim) can query any identity.
- IAM: only `GetItem`/`Query` on the baseline and alerts tables.

Done when: a request with no token gets 401, and a test with two student tokens shows A can never read B's alerts.

13 Dashboard frontend

BRANCH	DEPENDS ON	COST	OWNER
`task-13-dashboard-ui`	12	~$0	

- `dashboard/`: plain HTML, CSS and JS with no build step. It uses the Cognito PKCE login and sends the ID token to the API.
- A private S3 bucket behind CloudFront with OAC. Terraform uploads the files and generates `config.js`.
- Build the four screens shown on pages 2–5: My alerts, What's normal for me, What triggers an alert (with what-if), and Instructor view.

Done when: a student signs in and sees a new alert within about a minute of running an attack.

14 CLI: see your baseline with your aws-cli login

BRANCH	DEPENDS ON	COST	OWNER
`task-14-cli-baseline`	8	$0	

- `helpers/my_baseline.py`: `sts get-caller-identity` gets the ARN. Then a `GetItem` on the baseline, and print `explain_baseline`. A `--what-if` flag gives a predicted score.
- IAM policy `insider-threat-self-read`: read on both tables only when `dynamodb:LeadingKeys` = `arn:aws:iam::${aws:PrincipalAccount}:user/${aws:username}`.

Done when: rency sees their own baseline, and asking for dana's baseline returns AccessDenied.

15 Run attack scenarios from the dashboard

BRANCH	DEPENDS ON	COST	OWNER
`task-15-attack-runner`	12	$0	

- Move the scenarios out of `helpers/simulate_attack.py` into an importable module. The CLI must keep working.
- An `attack-runner` Lambda behind `POST /attacks/{scenario}`. It assumes the student's sandbox role, so CloudTrail records the attack under that student's identity.
- Limit each student to one run every 5 minutes. `mock-root` and `detector-off` are instructor-only.
- This Lambda stays **outside the VPC** even when `enable_vpc = true`. It calls IAM, STS and SSM and never touches the tables.

Done when: clicking "Run secret-theft" produces an alert above 75 in that student's list.

16 Student and instructor guides

BRANCH	DEPENDS ON	COST	OWNER
`task-16-guides`	13, 14	$0	

- `docs/STUDENT_GUIDE.md`: how to sign in, read an alert, use what-if and run a scenario, plus practice questions. Example: "Why did this score 160 and not 75?"
- `docs/INSTRUCTOR_GUIDE.md`: how to deploy (`bootstrap/` → `main/`), add students, choose `enable_vpc` and tear down at the end of term.

Done when: a classmate who has never seen the project can go from zero to their first alert using only the guide.